MarketPilot AI — Study Notes (Part 2)

Steps 4-7 · Side concepts · Interview prep

Devansh · 2026

MarketPilot AI — Study Notes (Part 2)

The continuation of NOTES.md. Part 1 covered the framework, the first 3 steps, the glossary, and the core interview answers. **Read Part 1 first.** Part 2 picks up at Step 4 and ends at the fully-shipped product.

Same structure as Part 1: read top to bottom once, then use as a reference.

Table of contents

1. Where we left off

By the end of Part 1, the agent could:

- Read websites via a real cheerio-based site connector.
- Run structured SEO audits and persist them.
- Route every tool call through the GREEN / YELLOW / RED tier gate.
- Refuse unknown tools by failing closed.

What was still missing: any way to actually *do* things in the world (writes), any way to see persisted audits visually, any way for the user to fill in their product profile, any way to launch a run from the UI, no scheduled jobs, no memory across runs, and most screens were placeholders.

Steps 4–7 fix all of that.

---

2. Step 4 — Approvals inbox + campaign brief modal

Why this step existed

A real platform produces 20 proposed actions a week, not 3. Burying them inside whichever skill run produced them doesn't scale — humans want one inbox. So Step 4 introduces a **dedicated Approvals screen** and a **pending-count badge** in the sidebar so you always know how many decisions are queued.

Step 4 also fixed a related missing piece: there was **no campaign brief form** in the UI. The `+ New Run` button did nothing. We built a modal that posts to `/api/agent/start`.

What we built (in code)

Backend:

- backend/src/routes/approvals.ts — `GET /api/approvals?status=...`, `GET /api/approvals/:id`, `POST /api/approvals/:id/decide`.
- `GET /api/approvals-count` — tiny endpoint that just returns `{ pending: N }`. Cheap to poll for the sidebar badge.
- In agent-loop.ts, when the agent calls `finish()`, every proposedAction it returns is **also mirrored as a real `Approval` record** in the approvals store. The `actionId` links the two so future code can keep them in sync.

Frontend:

- `ApprovalsView` — fetches `/api/approvals`, renders with filter pills (pending / approved / rejected / all), Approve / Reject buttons hit `/api/approvals/:id/decide`.
- `NavButton` now accepts an optional `badge` prop and renders a red pill with the count.
- `usePendingApprovalsCount` hook — polls every 5 seconds.
- `NewRunModal` — campaign brief form: skill picker (10 cards), product name, target audience, campaign goal (required), brand tone pills, main channel, budget, launch timeline. Submit posts to `/api/agent/start`, closes the modal, auto-navigates to the Approvals inbox.

Side concepts in this step

Polling. The frontend doesn't have a WebSocket or Server-Sent Events connection to the backend. Every few seconds it just re-fetches the endpoint. That's called "polling." It's not as fast as a live socket, but it's **trivial** to implement and works behind any firewall / proxy. For an inbox with < 50 items polling every 5 seconds is invisible to the user.

Why a dedicated count endpoint? We could ask "how many pending approvals?" by fetching the full list and counting. But that ships every approval's body across the network 12 times a minute. `/api/approvals-count` returns one number — tiny payload. Cheap to poll often.

The `void` operator. You'll see `void load()` in our `useEffect` blocks. `load()` returns a `Promise`. Without `void`, ESLint complains that we're creating a promise and ignoring it ("floating promise"). `void` is the explicit "I know, I don't want to wait for this." It does nothing at runtime — it's a marker for the reader and the linter.

Modal pattern. A modal is a UI overlay that grabs focus and dims the page behind it. Three details every modal needs to handle:

- **Backdrop click** → close (we use `onClick` on the outer div + `e.stopPropagation()` on the inner).
- **ESC key** → close (we add a `keydown` listener in `useEffect`, remove it on unmount).
- **Submit disabled until valid** — we compute `canSubmit` from the required fields and disable the button.

The `actionId` link between two stores. The agent's `finalReport.proposedActions` and the new `approvalsStore` both hold roughly the same data. Why both? Because each store has a different audience: the run's final report is shown inside the run, and the approvals store powers the global inbox. Linking them by a shared `actionId` lets future code keep them in sync without duplicating logic today.

What you'd say in an interview

> "Step 4 turned proposed actions from a side-effect of one run into a first-class queue. Every action the agent suggests now lands in the approvals store as its own record. The inbox is just

> a list view over that store with decide buttons. The sidebar badge polls a cheap count endpoint every 5 seconds — fail-graceful if the backend is down, the last value stays on screen."

---

3. Step 5 — First YELLOW write tool + the rollback contract

Why this step existed

Until Step 5 the agent could only *propose* things. Step 5 was about **making the agent actually do something** — and doing it in a way where rollback was trivially possible.

The framework rule: every write tool must return a `rollbackPayload` and implement a `rollback()` function. Tools that can't be rolled back are forced to RED tier forever (no auto-execute). So we needed a write tool where rollback was easy. The answer: **anything that goes through Git.**

A **Pull Request** is the perfect first write:

- The change isn't applied until you merge the PR.
- Closing the PR = canceling the change. That's rollback for free.
- The audit trail (who, what, when) is in Git already.

So we built `add_alt_text` — when the agent sees an "images missing alt-text" finding, it can open a PR to fix them.

What we built

Backend:

- backend/src/lib/connectors/types.ts — added `canFixAltText` capability + `AltTextPatch` type + `fixAltText` + `rollback` methods on the `SiteConnector` interface.
- backend/src/lib/connectors/github/mdx.ts — the GitHub-MDX connector. Initially **simulation only** (logged the PR contents, returned fake URL). In Step 7 we wired real Octokit calls when `GITHUB_TOKEN` is set.
- backend/src/lib/tools/seo/add-alt-text.ts — the tool body. Checks `capabilities.canFixAltText` before calling, returns `{ result, changeId, rollbackPayload }`.
- agent-tools.ts — registered `add_alt_text` (declaration, `toolMeta: YELLOW`, dispatch case). New exported `rollbackToolCall(toolName, payload)` function.
- agent-loop.ts — when a YELLOW tool returns a rollback payload, a `ToolCall` record is created in `tool-calls` store with status `executed`.

- backend/src/routes/tool-calls.ts — `GET /api/tool-calls`, `POST /api/tool-calls/:id/rollback`.

Side concepts in this step

Pull Request (PR). A Pull Request is Git's way of saying "I made these changes on a branch — please review and merge." The change isn't live on `main` until someone merges. So the PR itself is a **proposed change**. Closing it (without merging) is the same as deleting it. That's why PR-based writes have free rollback.

Branch / commit / SHA. Git tracks code in a tree of commits. A "branch" is just a movable pointer at some commit. A "SHA" is a unique fingerprint of a commit. When Octokit's `getRef` returns a `sha`, that's the unique ID of the commit at the tip of that branch.

Base64 encoding. When you upload file content through GitHub's API, you have to send it as base64 — a way to encode binary data as plain ASCII. That's what `Buffer.from(text).toString("base64")` does. GitHub stores the file as binary, then decodes when displaying. We use base64 because the API expects it; otherwise the JSON would break on special characters.

Personal Access Token (PAT). GitHub's quick way to authenticate without OAuth. You generate a token in GitHub Settings, give it to the app via env var, and it acts on your behalf. Less secure than OAuth (a leaked token has full scope) but **dead simple** for a single-user prototype. Real multi-user products move to OAuth. We deferred OAuth and used a PAT for now.

Simulation-first / real-second. We didn't write Octokit on day one. Instead, we built the connector with the **same interface** the real one will have, but the body just `console.log`'d the simulated PR. That let us prove the entire flow (gate → tool → store → rollback endpoint) without spending OAuth time. When Step 7 added real Octokit, **only the connector body changed** — everything else stayed identical. That's the value of stable interfaces.

Rollback payload contract. What goes in a rollback payload? Just enough info to reverse the write. For our PR write that's: `{ owner, repo, prNumber, branch, patches, simulated }`. The `simulated` flag lets the rollback code pick between calling `octokit.pulls.update({ state: "closed" })` (real) and just logging (simulated).

The `try/catch` + fallback pattern. In the connector, the real Octokit path is wrapped in `try/catch`. If anything throws — network error, expired token, repo doesn't exist — we log it and **fall through to the simulation**. The agent loop never crashes. This is called "graceful degradation": worse-quality behavior beats crashing.

What you'd say in an interview

> "Step 5 introduced the first real write — `add_alt_text`. It's YELLOW tier because rollback is built in: closing the Pull Request undoes the change. The connector has two paths — real Octokit calls when `GITHUB_TOKEN` is set, simulation otherwise. Same interface, same rollback payload shape, so the tool body never has to know which is which. Every YELLOW execution writes a `ToolCall` record with the rollback payload, and there's a `POST /api/tool-calls/:id/rollback` endpoint that re-invokes the connector's `rollback()`."

---

4. Step 6 — Memory + Scheduler

Why this step existed

Up to Step 5, every agent run started cold. It had no idea what the product was, what audits had already run, or how traffic was trending. So the agent kept asking the user questions the user had already answered (industry, voice, KPIs) or kept re-auditing pages that were audited yesterday.

The fix: **load memory before every run**. Read the workspace's product profile, recent audits, recent performance — and prepend it to the agent's prompt.

The other thing missing: **scheduled jobs**. The framework promised "weekly SEO audit every Monday 6am." That needed `node-cron`.

What we built

Backend:

- backend/src/lib/store/product-profile.ts — seeds a sensible default profile at module load. Now the agent always has *something* to read.
- Added fields to `ProductProfile`: `productName`, `tagline`, `industry`, `stage`, `siteUrl`, `features`, `differentiators`, `voiceTone` (string→array), `mrr`, `monthlyTraffic`, `northStar`.
- backend/src/routes/profile.ts — `GET /api/profile`, `PUT /api/profile`.
- backend/src/lib/memory/load.ts — `loadMemory()` bundles profile + recent audits + performance. `renderMemoryForPrompt()` turns the bundle into text for the LLM.
- agent-loop.ts — calls `loadMemory()` at the top of every run, writes a `memory_loaded` event, injects the rendered memory into the initial prompt.
- backend/src/scheduler/index.ts — job registry + `registerCronJobs()` called from `server.ts` at boot. Manual trigger via `POST /api/scheduler/run/:jobId`.
- backend/src/scheduler/weekly-seo-audit.ts — reads the profile's `siteUrl` and kicks off a `seo-audit` skill run with a sensible auto-generated brief.

Side concepts in this step

Cron expression format. `0 6 * * 1` means "minute 0, hour 6, any day-of-month, any month, day-of-week 1 (Monday)" — i.e. every Monday at 6:00 AM. Five fields, in this order: **minute · hour · day-of-month · month · day-of-week**. `*` means "every." `0` in a field means "exactly zero." Beginners often get this wrong because the fifth field's days-of-week starts at 0 (Sunday) or 1 (Monday) depending on the library. `node-cron` uses 0=Sunday, but `1` is unambiguously Monday.

Pure functions. `loadMemory()` and `renderMemoryForPrompt()` have zero side effects — they read inputs, return outputs, never modify anything. Pure functions are dead simple to unit-test: no mocks, no setup, no teardown. Just call with fake input, assert on output. Our test file `memory/__tests__/load.test.ts` runs in milliseconds with no network.

Why memory text instead of structured data? Gemini accepts function-calling schemas (structured tool inputs), but for the **prompt body** the LLM reads plain text. So we have to convert our nice `MemoryBundle` object into a multi-line string. `renderMemoryForPrompt()` is just an opinionated string builder. If empty (no audits yet), it cleanly skips that section instead of saying "0 audits" — saves tokens, looks honest.

Scheduler durability. `node-cron` runs **only while the server process is alive**. If your server restarts at 5:59 AM and the cron was 6:00, the job is missed entirely. Production schedulers (Inngest, Trigger.dev, Sidekiq, etc.) persist their schedule to a database and **catch up** when the worker comes back. We picked `node-cron` because it's literally one npm package and zero infrastructure. We documented the limitation in AGENTS.md §4 so it's not a hidden gotcha.

Manual-trigger endpoint as a dev affordance. Waiting until Monday 6am to test a cron is insane. `POST /api/scheduler/run/:jobId` just calls the same handler the cron would call. Same code, no wait. In production this endpoint would be admin-only; today it's open because we're single-user.

Why the user can edit the profile but the agent can't. Framework rule §5.12: **the agent never modifies the product profile.** This is a safety boundary — we don't want the agent rewriting "what your company is" based on something it read on Twitter. The profile is **user intent**. The agent reads it as constraints. The `PUT /api/profile` endpoint is called only by the frontend form, never by a tool.

What you'd say in an interview

> "Step 6 plugged two holes — no memory and no schedule. Memory is a pure function that bundles the profile, recent audits, and recent performance into a text block prepended to every prompt. The scheduler is `node-cron` registered at boot; there's a manual-trigger endpoint so I don't wait for Monday during demos. Both write events to the event log so the activity feed picks them up."

---

5. Step 7 — Finishing the visible product

Why this step existed

After Step 6, the back-end was solid but the front-end still had four placeholder screens (Agents, Drafts, Integrations, Settings), no Chat, and the dashboard was still mock data. Step 7 finished the picture.

It also did three architectural cleanups: per-skill tool manifests, real OAuth for GitHub, and the proper "rollbackable tool calls" management UI.

What we built

Backend:

- backend/src/routes/connections.ts — `GET / POST / PUT /api/connections`.
- backend/src/routes/chat.ts — `POST /api/chat`. Single-turn Gemini call with the memory bundle prepended.
- backend/src/routes/drafts.ts — `GET /api/drafts` flat gallery feed across all runs.
- New endpoints in server.ts: `GET /api/skill-runs`, `GET /api/dashboard-stats`, `GET /api/audits`, `GET /api/audits/:id`.
- backend/src/lib/skills/manifest.ts — per-skill tool allow-list.
- agent-loop.ts — filters `toolDeclarations` against the skill's allow-list before sending to Gemini.
- backend/src/lib/connectors/github/mdx.ts — **real** Octokit path for PR creation + close. Falls back to simulation if `GITHUB_TOKEN` is absent or the call throws.
- Seeded 7 default connections so the Connections screen has something to render.

Frontend (all in frontend/app/page.tsx):

- Rewrote `DashboardView` to read real data from `/api/dashboard-stats`, `/api/skill-runs`, `/api/events`, `/api/approvals`. Live runs strip shows the agent's current step. Event log feeds the activity panel with color-coded dots. Drafts panel pulls real drafts.
- `AgentsView` — list every skill run with status badge; click a row to expand and see the live trace.
- `DraftsView` — gallery of every draft saved by every run; filter pills; one-click copy to clipboard.
- `ConnectionsView` — 7 integration cards with status dots, config preview, help text.
- `SettingsView` — scheduler controls (Run-now buttons) + rollbackable tool calls list with **Roll back** buttons.
- `ChatWidget` — floating gradient button (bottom-right), opens a chat panel, sends to `/api/chat`, enter-to-send.

Side concepts in this step

`Promise.all`**.** When the dashboard hook fetches stats + runs + events + approvals, it fires all four requests **in parallel**, not sequentially. `Promise.all([a, b, c, d])` returns a single Promise that resolves when *all* are done. Total latency = the slowest of the four. If you `await` them one at a time, it's the sum. Big difference when each call is 100ms.

`navigator.clipboard.writeText`**.** Modern browser API for "copy this string to the clipboard." Used in the Drafts gallery's Copy button. Returns a Promise — we `void` it because we don't care to confirm.

Floating widget pattern. Why is Chat a floating bottom-right button instead of a sidebar item? Because it's a **reactive surface** — you might want to ask the assistant something while you're on the Audits page. A sidebar item would force you to leave whatever you're doing. Linear, Notion, Intercom all do this. The widget renders outside the main layout so it overlays everything.

Single-turn vs multi-turn vs streaming chat.

- **Single-turn** (what we have): each user message is independent. Backend doesn't remember the prior turns. Simple to implement, no chat history persistence needed.
- **Multi-turn**: backend stores conversation history and sends it back to the LLM each turn. Better answers, more cost (you re-send the history each call).
- **Streaming**: the LLM emits tokens as it generates them; the UI shows the answer growing. Lower perceived latency. Requires Server-Sent Events or chunked HTTP on the server side. We have neither today.

Per-skill tool manifest. Every agent run picks one skill. The skill should only see the tools it needs. A copywriting skill shouldn't have access to `add_alt_text`. So we ship a manifest mapping `skillId → string[]` of allowed tool names. The agent loop filters `toolDeclarations` before sending to Gemini. Skills not in the manifest fall back to a sensible default set.

Why filter, not enforce? Two layers of defense. The manifest controls what Gemini *sees* (so it doesn't even know `pause_ad_set` exists when running `copywriting`). The tier gate enforces at dispatch time (so even if Gemini hallucinated a tool name, it gets blocked). Defense in depth.

Status pills + colored dots. Pattern used everywhere — Connections, Approvals, Tool-calls, Audits. The trick is to define a `Record<Status, string>` mapping the status to its Tailwind class, then index into it. Avoids ifs and means new statuses just need a new line.

Polling intervals across the app. Dashboard polls every 4 seconds. Approvals every 5. Connections every 8. Drafts every 6. Why different? Frequency-of-change. The dashboard changes most (live runs ticking), so 4s. Connections almost never change — 8s is fine. There's no rule; tune to user expectation.

Real Octokit path with simulation fallback. When you set `GITHUB_TOKEN` + `GITHUB_OWNER` + `GITHUB_REPO`, the connector uses Octokit to:

1. `repos.get` → find the default branch.
2. `git.getRef` → get its head SHA.
3. `git.createRef` → make a new branch `agent/alt-text-<timestamp>`.
4. For each patch: `repos.getContent` → read the file → regex-add `alt=` to the right `<img>` → `repos.createOrUpdateFileContents` → commit.
5. `pulls.create` → open the PR.

If anything throws, we log + fall back to simulation. The agent never crashes; the user always gets a PR URL (real or fake).

What you'd say in an interview

> "Step 7 finished the surfaces. Every nav item is now a real screen wired to a real endpoint. The dashboard reads dashboard-stats, runs, events, and approvals in parallel via `Promise.all`. Settings has a Roll back button per tool call that hits the rollback endpoint built in Step 5. Chat is a floating widget — single-turn for now, prepends the memory bundle. And the GitHub connector graduated from simulated to real-with-fallback: if `GITHUB_TOKEN` is set, it opens actual PRs via Octokit; if not, it simulates. The agent loop also filters tool declarations through a per-skill manifest, so a copywriting skill literally can't see SEO write tools."

---

6. Side concepts you'll be asked about

A flat list, every term beginner-explained. Pull these into NOTES1's glossary mentally — both files are one big reference.

Polling

Asking the backend "any updates?" on a fixed interval. Easy to implement, slightly wasteful, perfectly fine for < 100 items. The alternative is WebSockets (true real-time push) or Server-Sent Events. We use polling because the cost of switching is high and the benefit is low at our scale.

`Promise.all`

JavaScript's "fire multiple async things in parallel, wait for all to finish." If one rejects, the whole thing rejects. Used in the dashboard hook so four fetches run simultaneously instead of waiting in line.

Pure function

A function whose output depends only on its inputs, with no side effects. `runAuditChecks(page) → findings[]` is pure. `audit_seo(url)` is NOT pure because it does I/O. Pure functions are dead simple to test.

Rollback payload

The opaque blob a write tool stashes so its operation can be reversed later. For `add_alt_text` it's `{ prNumber, branch, patches, simulated }`. For `pause_ad_set` (future) it'd be the previous campaign state. Tools own the contents; the loop and store just carry it through.

`void` operator

TypeScript / JavaScript shorthand for "I know this is a Promise, I don't await it on purpose." Doesn't do anything at runtime. Used in `useEffect` to call an async function without ESLint complaining about a floating Promise.

Backdrop / Modal

A modal is a popup that takes focus. The backdrop is the dimmed area behind it. Clicking the backdrop closes the modal — convention. `e.stopPropagation()` on the inner card prevents clicks INSIDE the modal from closing it.

`useEffect` with cleanup

React hook for side effects (network calls, timers, subscriptions). The function it returns is the **cleanup** — runs when the component unmounts or before the effect re-fires. Pattern in our polling effects: set up `setInterval`, return `() => clearInterval(t)`.

Personal Access Token (PAT)

GitHub's shortcut for "let this app act as me." A long string stored in env. Less safe than OAuth (leaking it is bad), but fine for solo-dev mode. Real products graduate to OAuth where users grant scoped access without sharing a token.

OAuth

The standard way for an app to act on a user's behalf without seeing their password. The user clicks "Connect," gets redirected to the provider, approves, and the provider hands the app a token. We use OAuth tokens internally; we just haven't built the redirect flows for GA4 / GSC / Meta Ads yet.

`Buffer.from(text).toString("base64")`

Convert a string to base64. GitHub's content API requires it. Base64 is a way to encode arbitrary bytes as ASCII letters — works in JSON, URLs, headers, anywhere.

Graceful degradation

If the real path fails (network error, token expired), fall back to a worse-but-working path. Better than crashing. Our GitHub connector does this — falls to simulation if Octokit throws.

Pull Request

A Git mechanism for proposing changes. A branch with commits → PR → review → merge or close. Closing without merging = the change is dropped. That's why PR-based writes are perfect for YELLOW: rollback is "close it."

Tier gate (recap)

A single function called before every tool dispatch. Decides GREEN (run), YELLOW (run + notify), or RED (block, create approval). Failing closed on unknowns is the safety net.

Capability declaration

Every connector exposes a `capabilities: { canCrawl, canWriteMeta, canFixAltText, writesViaPR, ... }` object. Tools check it before writing. A WordPress connector might have `canWriteMeta: true, canFixAltText: false`, and the corresponding tool returns an error rather than throwing.

Fail closed

When uncertain, refuse. Our tier gate sees an unknown tool name → treats it as RED. The alternative (fail open) would silently allow the call, which is a security/safety bug.

Stable interface

A function's input/output shape doesn't change even when the implementation does. `siteConnector.crawl(url): Promise<CrawledPage>` is stable — we could swap cheerio for Playwright tomorrow and nothing else in the codebase notices.

Repository pattern

Hide a data store behind a small set of methods (`create`, `get`, `list`, `update`). Callers don't know if it's a `Map`, SQLite, or Postgres underneath. All 9 of our stores follow this pattern.

Event log / event sourcing (lite)

An append-only list of "something happened" records. We don't fully event-source (we still mutate stores directly), but we **log** every gate decision, tool call, approval, rollback, scheduler tick. That log powers the Activity panel and is the ground truth for debugging.

Manifest

A small data file declaring "what is allowed for X." Our skill manifest declares which tools each skill can call. Other parts of the framework spec call for connector manifests (which integrations a skill needs).

Rate limit / 429

HTTP 429 = "you're sending too many requests." Gemini's free tier has daily quotas; if you blow them, you get 429s until reset. We saw this during testing. Production needs **exponential backoff** (wait 1s, then 2s, then 4s, then 8s) and request queuing. We don't have that yet — listed in NOTES §12.

Streaming response

Instead of waiting for the full reply, the server emits chunks as the LLM generates them. Lower perceived latency. Requires Server-Sent Events or chunked HTTP. Our chat doesn't stream yet; it sends one fat response.

`setInterval` and clearing it

`setInterval(fn, ms)` calls `fn` every `ms` milliseconds, returns an id. `clearInterval(id)` stops it. Polling = `setInterval` + cleanup in `useEffect` return.

`navigator.clipboard.writeText`

Browser API: "put this string in the clipboard." Used by the Drafts gallery Copy button. Requires HTTPS or `localhost` to work.

Dependency injection (lite)

Passing dependencies in rather than reaching for globals. Our tools receive an `AgentContext` with workspace id, logger, etc., so the tool doesn't `import store` directly. Easier to test, swap implementations, run in isolation.

JSON Schema

A schema language for describing JSON shapes. Gemini's function calling uses it: every tool declaration says "my input is `{ url: string }`" in JSON Schema. The LLM matches the shape; we validate before dispatching.

Microtask

JavaScript runs async callbacks (after `await`, `Promise.then`) in "microtasks" between synchronous chunks. Calling `setState` after an `await` runs in a microtask, which keeps React happy. Calling

`setState` *directly* inside a `useEffect` body is what the linter warned us about — we fixed it with `void load()`.

---

7. Every screen, one paragraph each

Dashboard

The home page. Greets you with how many agents are running and how many approvals are pending — both from the backend. Below: a strip of currently-running agent cards (with their current tool call as a status label), four real stat cards with sparklines, the latest 3 pending approvals, the latest 8 events from the log color-coded by type, and the latest 4 drafts the agent has saved. Auto-polls every 4 seconds. If the backend goes down, shows an "Offline · using stale data" pill in the top right but keeps showing the last known data.

Campaigns

The mock SEO trace mockup. We left it in because it's pretty, but the **real** version of this screen lives at **Agents** → click any run. Future cleanup item.

Agents

The list of every skill run that's ever happened in this workspace. Status badge (running with pulsing dot, completed, failed), skill id, product name, step count, draft count, relative time. Click a row to expand and see the live tool-call / tool-result trace in monospace. Polls every 4 seconds.

Drafts

Gallery of every draft saved by every run, across all skills. Filter pills at top (all + one per type — blog_outline, ad_copy, etc.). Each card shows type, time, title, first 400 chars, the skill that produced it, the product name, and a Copy button that puts the full content on the clipboard.

Proposed Actions

The approvals inbox. Filter pills (pending / approved / rejected / all), live count summary, cards with reasoning + impact + rollback plan, Approve / Reject buttons. Decided cards show their decided-at time. The sidebar item gets a red badge with the pending count.

SEO Reports

Audits history. Lists every audit the agent has saved, newest first. Each card shows the audited URL, big color-coded score (green ≥80, amber ≥60, rose <60), and severity counts (critical / warning / info). Click to expand and see every finding with severity badge, message, and stable check id.

Product Profile

The form. Loads from `/api/profile` on mount, edits a local `draft` copy, Save button hits `PUT /api/profile`. Top-right shows the live save state — "Unsaved changes" amber, "✓ Saved Xs ago" emerald. Profile Strength ring on the bottom-right is computed live from a 12-check list; missing items list updates as you fill in fields. The agent reads the saved profile in memory before every run.

Integrations

The Connections panel. 7 cards (Website, GA4, GSC, GitHub, Meta Ads, WordPress, Email). Each shows a status dot (active / pending / error), a short help line, and a config preview. Info banner at the bottom explains how to wire real OAuth.

Settings

Two sections. **Scheduled Jobs**: list of registered cron jobs with their cron expression and a "Run now" button that hits `POST /api/scheduler/run/:jobId`. **Rollbackable Tool Calls**: every YELLOW write the agent has done, with status, change ID (linked), and a "Roll back" button that hits `POST /api/tool-calls/:id/rollback`.

Floating Chat (every screen)

Bottom-right gradient button. Click to open a 400×560 panel. Type a question, press Enter, sends to `/api/chat`. The backend prepends the workspace memory bundle, calls Gemini once, returns the reply. User and assistant messages render in a familiar bubble layout.

---

8. The full endpoint list (updated)

Method	Path	Purpose
`GET`	`/health`	Liveness probe.
`POST`	`/api/agent/start`	Launch a skill run. Body: skill + brief. → `{ taskId }`
`GET`	`/api/agent/:taskId`	Poll one run.
`POST`	`/api/agent/:taskId/approve`	Legacy approve endpoint (still works; new code uses `/api/approvals`).
`GET`	`/api/skill-runs?status=`	Every run, optional filter.
`GET`	`/api/dashboard-stats`	Aggregated counters for the dashboard.
`GET`	`/api/profile`	Read product profile.
`PUT`	`/api/profile`	Save product profile.
`GET`	`/api/connections`	List integrations.
`POST`	`/api/connections`	Create one.
`PUT`	`/api/connections/:id`	Update one.
`GET`	`/api/approvals?status=`	Inbox feed.
`GET`	`/api/approvals/:id`	One approval.
`POST`	`/api/approvals/:id/decide`	Approve / reject.
`GET`	`/api/approvals-count`	Just `{ pending: N }`.
`GET`	`/api/audits`	Audits history.
`GET`	`/api/audits/:id`	One audit + all findings.
`GET`	`/api/tool-calls`	Every persisted tool call (YELLOW writes).
`GET`	`/api/tool-calls/:id`	One.
`POST`	`/api/tool-calls/:id/rollback`	Undo it.
`GET`	`/api/drafts`	Flat gallery of drafts across runs.
`POST`	`/api/chat`	Single-turn Gemini Q&A.
`GET`	`/api/events?limit=N`	Tail the event log.
`GET`	`/api/scheduler/jobs`	Registered cron jobs.
`POST`	`/api/scheduler/run/:jobId`	Fire one manually.

9. The full folder map (updated)

```
marketing-agent-platform/
├── AGENTS.md                          # project-wide rules
├── README.md                          # front door
├── NOTES.md                           # study notes part 1
├── NOTES2.md                          # study notes part 2  ← this file
├── DEMO.md                            # interview walkthrough
├── marketing-agent-framework.md       # canonical spec
├── frontend/
│   ├── AGENTS.md
│   └── app/
│       ├── page.tsx                   # ~3500 lines — every screen
│       ├── layout.tsx
│       └── globals.css
├── backend/
│   ├── AGENTS.md
│   └── src/
│       ├── server.ts                  # Express boot + scheduler.register + all routes
│       ├── routes/
│       │   ├── agent.ts               # /api/agent/*
│       │   ├── approvals.ts           # /api/approvals/*
│       │   ├── tool-calls.ts          # /api/tool-calls/*
│       │   ├── profile.ts             # /api/profile
│       │   ├── connections.ts         # /api/connections
│       │   ├── chat.ts                # /api/chat
│       │   └── drafts.ts              # /api/drafts
│       ├── lib/
│       │   ├── agent-loop.ts          # the agent loop (memory + filter + run)
│       │   ├── agent-tools.ts         # tool registry (decls + tier + dispatch)
│       │   ├── agent/
│       │   │   └── tier-gate.ts       # the gate function
│       │   ├── connectors/
│       │   │   ├── types.ts
│       │   │   ├── site/cheerio.ts
│       │   │   └── github/mdx.ts      # real Octokit + simulation fallback
│       │   ├── tools/seo/
│       │   │   ├── audit-checks.ts
│       │   │   └── add-alt-text.ts
│       │   ├── skills/
│       │   │   └── manifest.ts        # per-skill tool allow-list
│       │   ├── memory/load.ts         # memory bundler + prompt renderer
│       │   └── store/                 # 9 in-memory stores + types + barrel
│       └── scheduler/
│           ├── index.ts               # node-cron registration
│           └── weekly-seo-audit.ts
└── .agents/skills/                    # 41 SKILL.md files
```

10. Glossary — new terms only

(Terms already in NOTES.md §13 aren't repeated.)

Backdrop — the dimmed area behind a modal. Clicking it usually closes the modal.

Backoff (exponential) — wait an increasing amount of time between retries (1s, 2s, 4s, 8s) so you don't hammer a rate-limited service.

Barrel file — an `index.ts` that just re-exports from sibling files so importers can write `from "../store"` instead of `from "../store/skill-runs"`. We use them in `lib/store/`, `lib/connectors/`, `lib/agent/`.

Capability — a flag a connector exposes (`canCrawl`, `canWriteMeta`, etc.). Tools check before calling.

Chat history — the list of prior turns in a conversation. We don't persist it today; the chat is single-turn.

Cleanup function — the function `useEffect` returns. Runs on unmount or before re-fire. Where you `clearInterval`.

Cron expression — five fields (minute hour day-of-month month day-of-week) describing when to run.

Defense in depth — multiple layers of safety so no single bug is catastrophic. The tier gate + the manifest = two layers blocking unauthorized tool calls.

Floating Promise — a Promise no one is awaiting. Often a bug. `void` is the explicit "yes, I meant to do that."

Floating widget — a UI control rendered fixed-position so it's available regardless of which screen you're on. Our Chat button.

Graceful degradation — when the ideal path fails, fall back to a worse-but-working path. The Octokit → simulation fallback is this.

JSON Schema — schema language for JSON shapes. Gemini's function-calling tools use a JSON Schema dialect.

Manifest — declarative list of "what's allowed." Our skill-tools manifest.

Microtask — JavaScript's "between sync chunks, run these callbacks." `await` resumption runs in a microtask.

Octokit — GitHub's official Node SDK. We use `@octokit/rest` for the real PR path.

PAT (Personal Access Token) — GitHub authentication shortcut for solo use.

Polling — re-fetching on an interval to check for updates. Cheap, simple, behind any firewall. Inferior to true push (WebSockets) for high-frequency updates.

Pure function — output depends only on inputs, no side effects. Easy to test.

Rate limit — a quota a service enforces. 429 = "you hit it."

Real-with-fallback — implementation pattern where the real integration runs if credentials exist, simulation otherwise. Lets you ship the mechanism before the auth.

Rollback dispatcher — the function that, given a tool name + payload, calls the right connector's rollback. Lives in `rollbackToolCall()` in `agent-tools.ts`.

Simulation mode — running through the motions without making real external calls. The PR write was simulation-only until Step 7.

Single-turn chat — each chat message is independent; the backend doesn't remember prior turns. Cheap, simple, less context.

Skill manifest — see Manifest above.

Stable interface — function signatures don't change when implementation does.

Status dot — the small colored circle next to a status. Visual cue at a glance.

`void` **operator** — explicit fire-and-forget for Promises.

---

11. Interview questions + good answers (Part 2)

(Builds on the 15 answers in NOTES.md §14.)

Q: Walk me through what happens when I click "Launch a campaign."

A: Frontend opens a modal, you pick a skill, fill in the brief. On submit, `POST /api/agent/start` creates a `SkillRun` record and kicks the agent loop off in the background. The loop loads memory (profile + recent audits + recent performance) into the prompt, filters the tool declarations through the skill's manifest, then sends the brief to Gemini 2.5 Flash. For each tool Gemini wants to call, the tier gate intercepts — if it's GREEN, run; YELLOW, run + log a notify event + persist a `ToolCall` record with the rollback payload; RED, block and create an approval. When the agent calls `finish`, every proposedAction it returned is mirrored into the approvals store. Frontend polls and shows them in the inbox.

Q: How does the approvals inbox work?

A: It's a list view over `approvalsStore`. Two sources fill the store: (1) the agent loop's `finish()` handler, which mirrors every proposedAction, and (2) the tier gate, which creates an approval whenever a RED-tier tool gets blocked. The inbox polls `/api/approvals?status=pending` every 5 seconds and renders cards with Approve / Reject buttons that hit `/api/approvals/:id/decide`. The sidebar's red badge polls a separate cheap `/api/approvals-count` endpoint.

Q: How does rollback work for a PR-based write?

A: When the agent calls `add_alt_text`, the GitHub connector opens a real PR (or simulated one) and returns a rollback payload — `{ owner, repo, prNumber, branch, patches, simulated }`. The agent loop stores that payload in the `tool-calls` store with status `executed`. When the user clicks Roll back in Settings, `POST /api/tool-calls/:id/rollback` reads the payload, calls `rollbackToolCall(toolName, payload)`, which dispatches to the GitHub connector's `rollback("alt-text", payload)`, which calls `octokit.pulls.update({ state: "closed" })`. Tool call status flips to `rolled_back`. Event logged.

Q: Why simulate the GitHub PR before integrating for real?

A: To prove the mechanism end-to-end (gate → tool → store → rollback endpoint) without spending OAuth time. The connector has a stable interface — `fixAltText(patches) → WriteResult`. Whether the body opens a real PR or logs a simulated one doesn't matter to callers. When we wired Octokit later, only the connector body changed.

Q: How does the agent get context about my product?

A: A pure function called `loadMemory(workspaceId)` bundles: product profile (industry, voice, ICP, north star), recent audits (last 5), recent performance (last 7 days). The bundle is rendered to text by `renderMemoryForPrompt()` and prepended to the agent's initial prompt. The agent never writes the profile — only the user does, via the Product Profile form.

Q: What is the weekly cron job doing?

A: `weekly-seo-audit` reads the workspace's profile, picks up the `siteUrl`, kicks off a `seo-audit` run with a synthesized brief ("audit this URL, propose 2-3 fixes, reference recent audits if any"). Registered with `node-cron` at `0 6 * * 1` — every Monday 6am local time. A manual trigger endpoint exists for testing.

Q: Why filter tools per skill?

A: Defense in depth. The skill manifest controls what Gemini *sees* — a copywriting skill won't even be told `pause_ad_set` exists. The tier gate enforces at dispatch — if Gemini somehow asks for a tool it shouldn't, the gate blocks it. Two layers means a single mistake doesn't break safety.

Q: How does the Chat differ from the agent loop?

A: The agent loop is multi-step: it picks a tool, looks at the result, decides next. The Chat is single-turn: one Gemini call, no tools, just the memory bundle + the user's message → a text reply. It's for reactive Q&A ("summarize my last audit"), not for doing work.

Q: What would you build next?

A: Three things. (1) Replace in-memory `Map`s with `better-sqlite3` so a restart doesn't wipe everything — one file per store changes, the rest stays. (2) Real OAuth flows for GA4 / GSC / Meta Ads so the agent has real performance data in memory. (3) Streaming chat with Server-Sent Events so the assistant feels responsive.

Q: How would you handle a Gemini 429?

A: Exponential backoff with a queue. First retry after 1s, then 2s, 4s, 8s, give up at 30s. If the backoff window exceeds the user's patience, return a structured "rate-limited, try again in N seconds" error instead of timing out silently. For scheduled jobs (the cron), shift to a real durable scheduler like Inngest so retries are free.

Q: Walk me through how you'd add a new tool.

A: Pick a category folder under `lib/tools/<category>/`. Write the pure implementation (`addAltText(input) → result`). Add a declaration to `toolDeclarations` so Gemini knows about it. Add an entry to `toolMeta` with the tier — if it writes and you can't rollback, RED. Add a `case` in `executeTool`. If it writes, also branch in `rollbackToolCall()`. Add the tool name to the manifests of skills that should use it. Write a unit test that calls the tool with sample input and asserts the output shape. Type-check.

Q: Walk me through how you'd add a new screen.

A: New component in `page.tsx` (until we split). Fetch data from `/api/<resource>` in a `useEffect`. Render with the same design tokens — `rounded-2xl` cards, `bg-white`, slate borders, indigo accents. Add a `nav === "<id>"` case in `Home()` and route to it. If the data needs to live update, add a `setInterval` and clean it up. If the screen has actions (Approve, Roll back, Run now), POST to the corresponding endpoint and re-fetch.

Q: How would you scale this to multi-tenant?

A: Every store keys by `workspaceId` already (we just hard-code `DEFAULT_WORKSPACE_ID`). Add Clerk auth to the frontend. Add an Express middleware that pulls `workspaceId` from the auth context and attaches it to `req`. Replace `DEFAULT_WORKSPACE_ID` references in routes with `req.workspaceId`. The agent loop and tools don't change — they already take a workspaceId.

Q: What's the role of the event log?

A: It's the single source of truth for "something happened." Every gate decision, tool call, approval creation, rollback, scheduler tick, LLM call writes one event. The dashboard activity feed reads from it. Debugging issues post-hoc reads from it. Future analytics will read from it. Routes never `console.log` business data — they emit events. This is the foundation for an audit trail when we eventually deploy to a real environment.

Q: How do you keep the codebase navigable for someone new?

A: Three docs. Root `AGENTS.md` has the rules. `frontend/AGENTS.md` and `backend/AGENTS.md` scope them. The framework spec is canonical. The `NOTES.md` files are interview-prep / study. The README has the architecture diagram + how-to-extend. Inside the code, every file's top comment explains its purpose, every store has a single responsibility, and every tool follows the same shape (declaration → tier → dispatch → optional rollback). A new contributor can be productive in an hour.

12. Final "what's NOT built" list

The 7-step build plan is **complete**. Honest list of what's deliberately deferred:

1. **No real database** — `Map` only. Restart wipes everything.
2. **No auth** — single workspace, single user.
3. **OAuth for GA4 / GSC / Meta Ads / WordPress / Email** — connections seeded as "pending" with no auth flow yet.
4. **Durable scheduler** — `node-cron` doesn't survive process restart.
5. **Chat is single-turn + non-streaming**.
6. `page.tsx` **is one ~3500-line file** — should split into `views/` + `components/` per the frontend AGENTS.md.
7. **Campaigns screen still shows mock SEO trace** — real trace lives in Agents.
8. **No backoff / queue for Gemini 429s**.
9. **No deploy config** (Vercel / Render / Fly).
10. **No** `.skill.json` **per-skill input schemas** — the skill manifest only controls the tool set, not per-skill input shape.

None of these break the architecture. All are swap-when-needed.

---

13. How to demo every screen (step-by-step)

```
# Terminal 1
cd c:\Users\devan\OneDrive\Desktop\marketing-agent-platform\backend
npm run dev

# Terminal 2
cd c:\Users\devan\OneDrive\Desktop\marketing-agent-platform\frontend
npm run dev
```

Open http://localhost:3000.

1. Dashboard — point at "Good evening, Devansh." Sub-line shows live counts. Click the floating purple chat button at bottom-right, ask "summarize my workspace." Close.

2. Product Profile — sidebar. Form is populated. Edit the Tagline. Top-right shows "Unsaved changes" amber → click Save → "✓ Saved Xs ago" green. The Profile Strength ring updates as you fill fields.

3. + New Run — sidebar button. Pick SEO Audit, productName = "Demo", campaignGoal = "Audit https://example.com using audit_seo." Launch agents.

4. Proposed Actions — auto-navigated here. Wait ~30s. Pending approvals appear; sidebar shows a red badge. Filter pills work. Approve one — status flips, badge drops.

5. SEO Reports — sidebar. The audit from step 3 is here. Click to expand findings.

6. Agents — sidebar. The run from step 3 is listed with status `completed`. Click to expand and see the trace (`tool_call audit_seo`, `tool_result`, `tool_call finish`).

7. Drafts — sidebar. If the agent saved any drafts, they're here. Copy button works.

8. Integrations — sidebar. 7 connection cards. Website is `active`, GitHub is `pending` (unless you set `GITHUB_TOKEN`), others are `pending`.

9. Settings — sidebar. Two sections. Click **Run now** on `weekly-seo-audit` — fires another run. If a YELLOW write has run (force one via the DEMO.md script), click **Roll back** on it — status flips to `rolled_back`.

10. Floating Chat — bottom-right on every screen. Ask "what should I work on next?" — gets a context-aware reply from Gemini referencing your profile and recent audits.

The whole tour fits in 5 minutes.

End of Part 2

Combined with NOTES.md and DEMO.md, this is the full study + interview kit. Everything in code is explained somewhere in these three files plus the AGENTS.md set.

Last updated after the build plan completed.